

Projekt 2: STRIPS

Autorzy: Emilia Sarna, Kalina Rączka

Data: 9 kwietnia 2026

Wprowadzenie: cel i zakres projektu

Celem projektu było zaimplementowanie 3 problemów z co najmniej 50 stanami z dziedziny STRIPS, a następnie próba rozwiązań za pomocą algorytmu forward planning bez i z heurystykami. Oprócz tego zdefiniowano także podcele dla problemów i rozwiązano trzy problemy z podcelami, których rozwiązanie wymagało minimum 20 stanów akcji.

Wymagania projektowe

Zadania na 4 punkty:

1. Wybierz dziedzinę STRIPS i trzy przykładowe problemy z co najmniej 50 stanami, gdzie rozwiązanie składa się z minimum 4 instancji akcji. Zdefiniuj je za pomocą STRIPS_domain i Planning_problem z AIPython.
2. Spróbuj rozwiązać problem za pomocą metody forward planning (dla dużych problemów można zrobić timeout na 5 minut). Zanotuj znalezione rozwiązanie jako ciąg wykonywanych akcji.
3. Zaproponuj heurystykę do problemu. Opisz jak działa i dlaczego sądzisz, że będzie pomocna. Rozwiąż problem z heurystyką i zanotuj czas rozwiązania problemu z heurystyką (tu wymagane jest, aby rozwiązanie zostało znalezione).

Zadania na 6 punktów:

1. Wszystkie zadania z poprzedniej części.
2. Zdefiniuj podcele dla problemów (minimum dwa podcele do każdego problemu). Rozwiąż ponownie problem z podcelami, z heurystyką i bez, analogicznie jak w zadaniach na 4 punkty.

Zadania na 8 punktów:

1. Wszystkie zadania z poprzednich części.
2. Zdefiniuj i rozwiąż dodatkowo trzy problemy z podcelami, których rozwiązanie wymaga minimum 20 instancji akcji.

Implementacja problemów

W ramach projektu zostały zaimplementowane 3 następujące problemy za pomocą STRIPS_domain i Planning_Problem z AIPython: - blocksword4 - magicworld - dinner

Blocksword4

Problem: Uporządkowanie bloków na stołach poprzez przestawianie ich. - Bloki mogą być na innych blokach lub stołach. - Tylko bloki, na których nic nie leży, mogą być przestawiane.

```
def create_blocks_world_domain(blocks, tables):
    """
    Create a blocks world domain.
    Features:
    - on_<block>: location (block or table) - where the block is
    - clear_<obj>: {True, False} - whether obj has nothing on top
    """

    boolean = {True, False}
    blocks_set = set(blocks)
    tables_set = set(tables)
    blocks_and_tables = blocks_set | tables_set

    feature_domain_dict = {}

    # For each block, track what it's on
    for block in blocks_set:
        possible_locations = blocks_and_tables - {block}
        feature_domain_dict[f'on_{block}'] = possible_locations

    # For each block and table, whether it's clear (nothing on top)
    for obj in blocks_and_tables:
        feature_domain_dict[f'clear_{obj}'] = boolean

    # Create move actions
```

```

actions = set()

# Move block X from Y to Z
# Preconditions:
#   - X is on Y
#   - X is clear
#   - Z is clear
# Effects:
#   - X is on Z
#   - Y is clear
#   - Z is not clear

for x in blocks_set:
    for y in blocks_and_tables:
        if x == y:
            continue
        for z in blocks_and_tables:
            if z == x or z == y:
                continue

            action_name = f'move_{x}_from_{y}_to_{z}'
            preconds = {
                f'on_{x}': y,
                f'clear_{x}': True,
                f'clear_{z}': True,
            }
            effects = {
                f'on_{x}': z,
                f'clear_{y}': True,
                f'clear_{z}': False,
            }
            actions.add(Strips(action_name, preconds, effects))

return STRIPS_domain(feature_domain_dict, actions)

def blocksworld_problem():
    """
    PROBLEM (Complex - 8 blocks, 4 tables)
    Initial: Blocks stacked on t1 and t2
    Goal: Different arrangement across 4 tables
    - Complex multi-goal arrangement
    """
    blocks = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']

```

```

tables = ['t1', 't2', 't3', 't4']
domain = create_blocks_world_domain(blocks, tables)

initial_state = {
    'on_a': 't1',
    'on_b': 'a',
    'on_c': 'b',
    'on_d': 'c',
    'on_e': 't2',
    'on_f': 'e',
    'on_g': 'f',
    'on_h': 'g',
    'clear_a': False,
    'clear_b': False,
    'clear_c': False,
    'clear_d': True,
    'clear_e': False,
    'clear_f': False,
    'clear_g': False,
    'clear_h': True,
    'clear_t1': False,
    'clear_t2': False,
    'clear_t3': True,
    'clear_t4': True,
}

goal = {
    'on_a': 't1',
    'on_b': 'a',
    'on_c': 't2',
    'on_d': 'c',
    'on_e': 't3',
    'on_f': 'e',
    'on_g': 't4',
    'on_h': 'g',
}

return Planning_problem(domain, initial_state, goal)

```

Magic World

Problem: Osiągnięcie transformacji magicznej poprzez zaklęcia i eliksiry. - Zbieranie składników
- Tworzenie eliksirów, by osiągnąć pożądane efekty. - Rzucanie zaklęć w celu transformacji obiektów. - Złamanie zaklęć - Stwarzanie zaklętych przedmiotów

```
def create_magicworld_domain():
    """
    Features:
    - have_ingredient: {True, False} - whether we have ingredient
    - have_potion: {True, False} - whether we brewed the potion
    - cast_spell: {True, False} - whether spell was cast
    - <object>_state: {normal, enchanted, petrified, invisible} - state of
    ↪ object
    - player_level: {apprentice, journeyman, master} - wizard skill level
    - mana: {0..100} - wizard's magical energy
    """

    boolean = {True, False}
    object_states = {'normal', 'enchanted', 'petrified', 'invisible',
    ↪ 'transmuted'}
    skill_levels = {'apprentice', 'journeyman', 'master'}

    feature_domain_dict = {
        # Ingredients
        'have_moonstone': boolean,
        'have_dragon_scale': boolean,
        'have_phoenix_feather': boolean,
        'have_crystal': boolean,
        'have_herbs': boolean,
        'have_unicorn_horn': boolean,

        # Potions (created from ingredients)
        'brewed_invisibility_potion': boolean,
        'brewed_strength_potion': boolean,
        'brewed_transformation_potion': boolean,
        'brewed_healing_potion': boolean,

        # Spells cast
        'cast_fireball': boolean,
        'cast_freeze': boolean,
        'cast_lightning': boolean,
        'cast_transmute': boolean,
```

```

# Object states
'sword_state': object_states,
'amulet_state': object_states,
'ring_state': object_states,

# Wizard status
'player_level': skill_levels,
'mana': {0, 25, 50, 75, 100},

# Additional flags
'magic_circle_drawn': boolean,
'ritual_prepared': boolean,
}

actions = set()

# ===== GATHER INGREDIENTS =====
# These actions acquire raw materials (no preconditions)
actions.add(Strips('gather_moonstone',
                   {},
                   {'have_moonstone': True})))

actions.add(Strips('gather_dragon_scale',
                   {},
                   {'have_dragon_scale': True})))

actions.add(Strips('gather_phoenix_feather',
                   {},
                   {'have_phoenix_feather': True})))

actions.add(Strips('gather_crystal',
                   {},
                   {'have_crystal': True})))

actions.add(Strips('gather_herbs',
                   {},
                   {'have_herbs': True})))

actions.add(Strips('gather_unicorn_horn',
                   {},
                   {'have_unicorn_horn': True})))

```

```

# ===== BREW POTIONS =====
# Create potions from ingredients

actions.add(Strips('brew_invisibility_potion',
    {'have_phoenix_feather': True, 'have_herbs': True},
    {'brewed_invisibility_potion': True,
     'have_phoenix_feather': False, 'have_herbs': False}))

actions.add(Strips('brew_strength_potion',
    {'have_dragon_scale': True, 'have_crystal': True},
    {'brewed_strength_potion': True,
     'have_dragon_scale': False, 'have_crystal': False}))

actions.add(Strips('brew_transformation_potion',
    {'have_unicorn_horn': True, 'have_moonstone': True},
    {'brewed_transformation_potion': True,
     'have_unicorn_horn': False, 'have_moonstone':
     ↪ False}))

actions.add(Strips('brew_healing_potion',
    {'have_herbs': True, 'have_crystal': True},
    {'brewed_healing_potion': True,
     'have_herbs': False, 'have_crystal': False}))

# ===== PREPARATION SPELLS =====
# These prepare the environment

actions.add(Strips('draw_magic_circle',
    {},
    {'magic_circle_drawn': True}))

actions.add(Strips('prepare_ritual',
    {'magic_circle_drawn': True},
    {'ritual_prepared': True}))

# ===== TRAIN TO INCREASE SKILL =====
actions.add(Strips('train_to_journeyman',
    {'player_level': 'apprentice', 'mana': 50},
    {'player_level': 'journeyman'}))

actions.add(Strips('train_to_master',
    {'player_level': 'journeyman', 'mana': 100},
    {'player_level': 'master'}))

```

```

# ===== RESTORE MANA =====
actions.add(Strips('meditate_restore_25',
                  {'mana': 0},
                  {'mana': 25}))

actions.add(Strips('meditate_restore_50',
                  {'mana': 25},
                  {'mana': 50}))

actions.add(Strips('meditate_restore_75',
                  {'mana': 50},
                  {'mana': 75}))

actions.add(Strips('meditate_restore_100',
                  {'mana': 75},
                  {'mana': 100}))

# ===== CAST SPELLS =====
# Spells require mana and have effects

actions.add(Strips('cast_fireball',
                  {'mana': 50, 'ritual_prepared': True},
                  {'cast_fireball': True, 'mana': 25}))

actions.add(Strips('cast_freeze',
                  {'mana': 50, 'ritual_prepared': True},
                  {'cast_freeze': True, 'mana': 25}))

actions.add(Strips('cast_lightning',
                  {'mana': 75, 'player_level': 'master'},
                  {'cast_lightning': True, 'mana': 50}))

actions.add(Strips('cast_transmute',
                  {'mana': 100, 'player_level': 'master',
↪  'ritual_prepared': True},
                  {'cast_transmute': True, 'mana': 50}))

# ===== ENCHANT OBJECTS =====
# Use potions and spells to enchant objects

actions.add(Strips('enchant_sword_with_invisibility',
                  {'sword_state': 'normal',
↪  'brewed_invisibility_potion': True, 'cast_freeze': True},

```



```

        {'sword_state': 'invisible',
↪  'brewed_invisibility_potion': False}))

    actions.add(Strips('enchant_sword_with_strength',
        {'sword_state': 'normal', 'brewed_strength_potion':
↪  True, 'cast_fireball': True},
        {'sword_state': 'enchanted', 'brewed_strength_potion':
↪  False}))

    actions.add(Strips('enchant_amulet_with_protection',
        {'amulet_state': 'normal', 'brewed_healing_potion':
↪  True, 'ritual_prepared': True},
        {'amulet_state': 'enchanted', 'brewed_healing_potion':
↪  False}))

    actions.add(Strips('enchant_ring_with_transformation',
        {'ring_state': 'normal',
↪  'brewed_transformation_potion': True, 'cast_transmute': True},
        {'ring_state': 'transmuted',
↪  'brewed_transformation_potion': False}))

    # ===== DISPEL MAGIC =====
    actions.add(Strips('dispel_sword_magic',
        {'sword_state': 'enchanted'},
        {'sword_state': 'normal'}))

    actions.add(Strips('remove_invisibility_from_sword',
        {'sword_state': 'invisible'},
        {'sword_state': 'normal'}))

    return STRIPS_domain(feature_domain_dict, actions)

def magicworld_problem():
    """
    PROBLEM
    Goal: Complex magical transformation
    - Enchant sword, amulet, and ring
    - Requires reaching master skill level
    - Multiple spell types needed
    """
    domain = create_magicworld_domain()

```

```

initial_state = {
    'have_moonstone': False,
    'have_dragon_scale': False,
    'have_phoenix_feather': False,
    'have_crystal': False,
    'have_herbs': False,
    'have_unicorn_horn': False,
    'brewed_invisibility_potion': False,
    'brewed_strength_potion': False,
    'brewed_transformation_potion': False,
    'brewed_healing_potion': False,
    'cast_fireball': False,
    'cast_freeze': False,
    'cast_lightning': False,
    'cast_transmute': False,
    'sword_state': 'normal',
    'amulet_state': 'normal',
    'ring_state': 'normal',
    'player_level': 'apprentice',
    'mana': 0,
    'magic_circle_drawn': False,
    'ritual_prepared': False,
}

goal = {
    'sword_state': 'enchanted',
    'amulet_state': 'enchanted',
    'ring_state': 'transmuted',
    'player_level': 'master',
    'mana': 50,
}

return Planning_problem(domain, initial_state, goal)

```

Dinner

Problem: Przygotowanie obiadu dla gości - Kupienie składników. - Przygotowanie artykułów. - Ugotowanie posiłków. - Nakrycie do stołu. - Zaproszenie gości.

```

def create_dinner_domain():
    """

```

```

Features:
- have_<ingredient>: {True, False} - whether we have the ingredient
- prepared_<item>: {True, False} - whether item is prepared
- cooked_<meal>: {True, False} - whether meal is cooked
- table_set: {True, False} - whether table is set
- <guest>_invited: {True, False} - whether guest is invited
- <guest>_present: {True, False} - whether guest has arrived
"""

boolean = {True, False}

feature_domain_dict = {
    # Ingredients
    'have_chicken': boolean,
    'have_rice': boolean,
    'have_vegetables': boolean,
    'have_wine': boolean,
    'have_dessert_ingredients': boolean,

    # Preparation
    'prepared_chicken': boolean,
    'prepared_vegetables': boolean,
    'prepared_rice': boolean,
    'prepared_dessert': boolean,

    # Cooking
    'cooked_chicken': boolean,
    'cooked_rice': boolean,
    'cooked_dessert': boolean,

    # Setup
    'table_set': boolean,
    'wine_opened': boolean,

    # Guests status (3 guests)
    'alice_invited': boolean,
    'bob_invited': boolean,
    'carol_invited': boolean,
    'alice_present': boolean,
    'bob_present': boolean,
    'carol_present': boolean,
}

```

```

# Actions
actions = set()

# SHOP actions - buy ingredients
actions.add(Strips('shop_chicken',
                   {},
                   {'have_chicken': True}))

actions.add(Strips('shop_rice',
                   {},
                   {'have_rice': True}))

actions.add(Strips('shop_vegetables',
                   {},
                   {'have_vegetables': True}))

actions.add(Strips('shop_wine',
                   {},
                   {'have_wine': True}))

actions.add(Strips('shop_dessert_ingredients',
                   {},
                   {'have_dessert_ingredients': True}))

# PREPARE actions - prepare raw ingredients
actions.add(Strips('prepare_chicken',
                   {'have_chicken': True},
                   {'prepared_chicken': True, 'have_chicken': False}))

actions.add(Strips('prepare_vegetables',
                   {'have_vegetables': True},
                   {'prepared_vegetables': True, 'have_vegetables':
↵ False}))

actions.add(Strips('prepare_rice',
                   {'have_rice': True},
                   {'prepared_rice': True, 'have_rice': False}))

actions.add(Strips('prepare_dessert',
                   {'have_dessert_ingredients': True},
                   {'prepared_dessert': True, 'have_dessert_ingredients':
↵ False}))

```

```

# COOK actions - transform prepared into cooked
actions.add(Strips('cook_chicken',
    {'prepared_chicken': True},
    {'cooked_chicken': True}))

actions.add(Strips('cook_rice',
    {'prepared_rice': True},
    {'cooked_rice': True}))

actions.add(Strips('cook_dessert',
    {'prepared_dessert': True},
    {'cooked_dessert': True}))

# SETUP actions
actions.add(Strips('set_table',
    {},
    {'table_set': True}))

actions.add(Strips('open_wine',
    {'have_wine': True},
    {'wine_opened': True}))

# INVITE actions
actions.add(Strips('invite_alice',
    {},
    {'alice_invited': True}))

actions.add(Strips('invite_bob',
    {},
    {'bob_invited': True}))

actions.add(Strips('invite_carol',
    {},
    {'carol_invited': True}))

# GUEST ARRIVAL actions - guest arrives if invited
actions.add(Strips('alice_arrives',
    {'alice_invited': True, 'cooked_chicken': True,
     'cooked_rice': True, 'cooked_dessert': True,
     ⇨ 'table_set': True},
    {'alice_present': True}))

actions.add(Strips('bob_arrives',

```

```

        {'bob_invited': True, 'cooked_chicken': True,
         'cooked_rice': True, 'cooked_dessert': True,
         ⇨ 'table_set': True},
        {'bob_present': True}))

    actions.add(Strips('carol_arrives',
                       {'carol_invited': True, 'cooked_chicken': True,
                        'cooked_rice': True, 'cooked_dessert': True,
                        ⇨ 'table_set': True},
                       {'carol_present': True}))

    return STRIPS_domain(feature_domain_dict, actions)

def dinner_problem_1():
    """
    PROBLEM
    Goal: Prepare simple dinner and have one guest arrive
    - Cook chicken and rice
    - Set table
    - Invite and have Alice arrive
    """
    domain = create_dinner_domain()

    initial_state = {
        'have_chicken': False,
        'have_rice': False,
        'have_vegetables': False,
        'have_wine': False,
        'have_dessert_ingredients': False,
        'prepared_chicken': False,
        'prepared_vegetables': False,
        'prepared_rice': False,
        'prepared_dessert': False,
        'cooked_chicken': False,
        'cooked_rice': False,
        'cooked_dessert': False,
        'table_set': False,
        'wine_opened': False,
        'alice_invited': False,
        'bob_invited': False,
        'carol_invited': False,
        'alice_present': False,
        'bob_present': False,
    }

```

```

        'carol_present': False,
    }

    goal = {
        'cooked_chicken': True,
        'cooked_rice': True,
        'cooked_dessert': True,
        'table_set': True,
        'alice_present': True,
    }

    return Planning_problem(domain, initial_state, goal)

```

Następnie zaimplementowany został algorytm **forward planning**: polega on przeszukiwaniu grafu stanów od stanu początkowego, aż do osiągnięcia zamierzonego celu. Wykorzystuje on przeszukiwanie wszerz - BFS oraz obsługuje timeout, by nie dopuścić do nieskończenia długiego poszukiwania rozwiązania.

```

def forward_planner_bfs(problem, timeout=120):
    """
    Forward planning using BFS (no heuristic)
    Returns: (actions, elapsed_time, expanded_nodes) or (None, elapsed_time,
    ↪ expanded_nodes)
    """
    start_time = time.time()
    initial = State(problem.initial_state)

    # Check if initial state is goal
    if all(initial.assignment.get(k) == v for k, v in problem.goal.items()):
        return [], 0.0, 0

    queue = deque([(initial, [])])
    visited = {initial}
    expanded = 0

    while queue:
        if time.time() - start_time > timeout:
            return None, timeout, expanded

        state, actions = queue.popleft()
        expanded += 1

```

```

# Generate successors
for action in problem.prob_domain.actions:
    # Check if action is applicable
    if not all(state.assignment.get(k) == v for k, v in
        ↪ action.preconds.items()):
        continue

    # Apply action effects
    new_assign = state.assignment.copy()
    new_assign.update(action.effects)
    new_state = State(new_assign)

    if new_state not in visited:
        new_actions = actions + [action]

        # Check if goal is reached
        if all(new_state.assignment.get(k) == v for k, v in
            ↪ problem.goal.items()):
            elapsed = time.time() - start_time
            return new_actions, elapsed, expanded

        visited.add(new_state)
        queue.append((new_state, new_actions))

elapsed = time.time() - start_time
return None, elapsed, expanded

```

Dzięki temu algorytmowi udało się znaleźć rozwiązania:

- dinner - 12 kroków w 1.3231s - Akcje: set_table, invite_alice, shop_chicken, prepare_chicken, cook_chicken, shop_dessert_ingredients, prepare_dessert, cook_dessert, shop_rice, prepare_rice, cook_rice, alice_arrives
- magicworld - 7 kroków w 0.0812s - Akcje: gather_herbs, gather_phoenix_feather, draw_magic_circle, prepare_ritual, cast_freeze, brew_invisibility_potion, enchant_sword_with_invisibility
- blocksword - 9 kroków w 48.3975s - Akcje: move_h_from_g_to_t3, move_g_from_f_to_t4, move_h_from_t3_to_g, move_d_from_c_to_h, move_f_from_e_to_c, move_e_from_t2_to_t3, move_f_from_c_to_e, move_c_from_b_to_t2, move_d_from_h_to_c

Heurystyki

Heurystyki to funkcje szacujące koszt lub dystans pozostały do osiągnięcia celu, które stosuje się jako „kompas” dla algorytmu, aby zamiast sprawdzać miliony przypadkowych stanów, skupił się on na tych najbardziej obiecujących. Dla każdego problemu zaimplementowano osobną heurystykę.

Blocksworld

Heurystyka stworzona do blocksworlda polega na liczeniu, ile bloków nie jest jeszcze w swoich docelowych pozycjach, gdy natomiast blok nie jest na swojej pozycji i ma na sobie inny blok (nie jest “clear”) to dodajemy jeszcze jeden ruch, który jest potrzebny. Ta heurystyka sprawia, że algorytm częściej będzie przestawiał bloki, które prowadzą do obniżenia ilości kroków szacowanych przez heurystykę.

```
def heuristic_blocksworld(state_dict, goal_dict):
    blocked_count = 0

    for block_key, goal_loc in goal_dict.items():
        if block_key in state_dict:
            if state_dict[block_key] != goal_loc:
                blocked_count += 1

            clear_key = block_key.replace('on_', 'clear_')
            if clear_key in state_dict and not state_dict[clear_key]:
                blocked_count += 1

    return blocked_count
```

Magicworld

Heurystyka stworzona dla magicworld oblicza, ile stanów końcowych nie jest spełnionych oraz dodatkowo karze za niższy poziom czarnoksiężnika w porównaniu do docelowego poziomu. Proste zliczanie celów do osiągnięcia jest dobrą i dopuszczalną heurystyką - nie przeszacowuje, a jednocześnie zmusza algorytm do osiągania celów i zmniejszania jej wartości. Poza tym, dodano do heurystyki szacowanie kroków do osiągnięcia potrzebnego poziomu gracza - zbliża to wartość heurystyki do faktycznej ilości kroków, które są potrzebne.

```
def heuristic_magicworld(state, goal):
    """
    Heuristic for magicworld:
```

```

Count missing goal facts.
Also consider wizard level - lower level = higher heuristic.
"""
missing = 0
for g in goal:
    if g not in state or state[g] != goal[g]:
        missing += 1

# Add extra cost if need to level up
if 'player_level' in goal and 'player_level' in state:
    level_cost = 0
    goal_level = goal['player_level']
    current_level = state['player_level']
    if goal_level == 'journeyman' and current_level == 'apprentice':
        level_cost = 2
    elif goal_level == 'master' and current_level == 'journeyman':
        level_cost = 2
    elif goal_level == 'master' and current_level == 'apprentice':
        level_cost = 4
    missing += level_cost

return missing

```

Dinner

Dla problemu dinner heurystyka liczy, ile końcowych faktów nie jest jeszcze usatysfakcjonowanych. Jest bardzo prosta, ale prowadzi do tego, żeby kolejne potrzebne stany zostały spełniane - prowadzi do celu po jednej zmianie oraz sprawia, że nie przeszukujemy stanów, które nie prowadzą do chociaż jednego spełnionego warunku.

```

def heuristic_dinner(state, goal):
    return len([g for g in goal if g not in state or state[g] != goal[g]])

```

Porównanie wyników: bez vs z heurystyką

Dla prawie wszystkich problemów wariant algorytmu z heurystyką okazał się szybszy niż bez niej (dla najmniejszych i najłatwiejszych problemów był bliski, a dla `magicworld_1` okazał się nawet lepszy ze względu na swoją prostotę). Największa poprawa była w przypadku najtrudniejszych problemów ze względu na dużą liczbę możliwych kroków w każdym ruchu, np. w `blocksworld_3`, gdzie różnica sięgnęła ponad 224 sekund.

Wniosek: Heurystyki przyspieszają znacznie czas rozwiązywania problemu - im więcej jest możliwości ruchu w każdym momencie, tym bardziej polepszają one wynik. Natomiast w żadnym z badanych przypadków nie zmieniła się liczba kroków - wynika to z charakterystyki użytego algorytmu Forward Planning - BFS zawsze znajduje najkrótszą drogę, ponieważ przeszukuje wszystkie możliwości od tych najkrótszych (jeśli oczywiście wszystkie ktoki mają tą samą wartość ("długość")).

Tabela 1: Porównanie algorytmu z i bez heurystyki.

Prob- lem	Liczba cech	Kroki bez heurystyki	Kroki z heurystyką	Czas bez heurystyki	Czas z heurystyką
dinner_1	20	12	12	0.785s	0.385s
dinner_2	20	16	16	1.575s	1.099s
dinner_3	20	18	18	1.720s	1.469s
magicword_1	20	7	7	0.033s	0.071s
magicword_2	20	11	11	0.416s	0.346s
magicword_3	20	23	23	5.920s	3.020s
blocksworld_1	20	2	2	0.000s	0.000s
blocksworld_2	20	5	5	0.013s	0.002s
blocksworld_3	20	9	9	24.583s	0.044s

Podcele

Są to pojedyncze własności stanu, które muszą zostać spełnione podczas drogi do ostatecznego celu. Zarówno korzystając z heurystyki, jak i bez kolejno szukamy kroków do osiągnięcia kolejnych podceli, a na końcu do osiągnięcia ostatecznego celu.

Dinner

Dla problemu dinner zostały zaimplementowane następujące podcele:

```
def get_dinner_subgoals():
    return {
        'dinner_1': [
            {'prepared_chicken': True, 'prepared_rice': True,
↪ 'prepared_dessert': True},
            {'cooked_chicken': True, 'cooked_rice': True, 'cooked_dessert':
↪ True},
        ],
        'dinner_2': [
            {'prepared_chicken': True, 'prepared_rice': True,
↪ 'prepared_dessert': True},
```

```

        {'cooked_chicken': True, 'cooked_rice': True},
        {'table_set': True, 'wine_opened': True},
        {'cooked_chicken': True, 'cooked_rice': True},
    ],
    'dinner_3': [
        {'have_chicken': True, 'have_rice': True,
↪ 'have_dessert_ingredients': True},
        {'prepared_chicken': True, 'prepared_rice': True,
↪ 'prepared_dessert': True},
        {'cooked_chicken': True, 'cooked_rice': True, 'cooked_dessert':
↪ True},
        {'table_set': True, 'wine_opened': True},
        {'have_chicken': True, 'have_rice': True,
↪ 'have_dessert_ingredients': True},
        {'prepared_chicken': True, 'prepared_rice': True,
↪ 'prepared_dessert': True},
    ],
}

```

Magicworld

Dla problemu magicworld zostały zdefiniowane następujące podcele:

```

def get_magicworld_subgoals():
    return {
        'magicworld_1': [
            {'have_phoenix_feather': True, 'have_herbs': True},
            {'brewed_invisibility_potion': True, 'cast_freeze': True}
        ],
        'magicworld_2': [
            {'have_phoenix_feather': True, 'have_herbs': True,
            'have_dragon_scale': True, 'have_crystal': True},
            {'brewed_invisibility_potion': True, 'brewed_strength_potion':
↪ True},
            {'cast_fireball': True, 'cast_freeze': True},
        ],
        'magicworld_3': [
            {'player_level': 'master'},
            {'mana': 100},
            {'ritual_prepared': True,
            'brewed_transformation_potion': True,

```

```

        'brewed_healing_potion': True},
        {'cast_fireball': True, 'cast_freeze': True},
    ],
}

```

Blocksworld

Dla blocksworld zostały zaimplementowane następujące podproblemy:

```

def get_blocksworld_subgoals():
    return {
        'blocksworld_1': [
            {'clear_b': True},
            {'on_b': 't3'}],
        'blocksworld_2': [
            {'clear_a': True, 'clear_c': True},
            {'on_a': 'b', 'on_c': 'd'}],
        'blocksworld_3': [
            {'clear_d': True, 'clear_h': True},
            {'on_d': 't1', 'on_h': 't2', 'on_b': 't3'}],
    }

```

Wyniki

Tabela 2: Porównanie działania algorytmów w wersji problemów z podcelami

Problem	Kroki bez heurystyki	Czas bez heurystyki	Kroki z heurystyką	Czas z heurystyką
dinner_1_subgoals	12	0.0277s	12	0.0152s
dinner_2_subgoals	16	0.0342s	16	0.0197s
dinner_3_subgoals	21	0.0164s	21	0.0109s
magicworld_1_subgoals	7	0.0026s	7	0.0033s
magicworld_2_subgoals	16	0.0245s	16	0.0172s
magicworld_3_subgoals	25	0.1016s	25	0.0510s
blocksworld_1_subgoals	4	0.0002s	4	0.0003s
blocksworld_2_subgoals	8	0.0224s	8	0.0024s
blocksworld_3_subgoals	22	198.0890s	22	4.2777s

Problem	Kroki bez heurystyki	Czas bez heurystyki	Kroki z heurystyką	Czas z heurystyką
---------	-------------------------	------------------------	-----------------------	----------------------

W dalszym ciągu korzystanie z heurystyk znacznie przyspiesza wynik - w przypadku najtrudniejszego problemu - `blocksworld_3` prawie 50-krotnie. W dalszym ciągu prawdziwa jest reguła - im więcej rozgałęzień w decyzjach który krok zrobił, tym ważniejsze jest użycie heurystyk - “prowadzi” ona przeszukiwanie stanów w dobrą stronę, znacznie zwiększając prawdopodobieństwo znalezienia jak najkrótszej drogi. Jednak krótszy czas przeszukiwania ma swoją cenę - heurystyka nie jest w stanie zapewnić znalezienia optymalnego, tego najkrótszego rozwiązania - również widać to w przypadku `blocksworld_3` - znalezione rozwiązanie w algorytmie z heurystyką jest dłuższe o 2 kroki.

Da się również zauważyć, że nie zawsze dodanie podcelów zwiększa ilość potrzebnych kroków do znalezienia rozwiązania - tak jak w przypadku `magicworld_1`, najprawdopodobniej określone podcele są niezbędnymi krokami do uzyskania ostatecznego stanu, więc nie potrzebne są dodatkowe kroki. Zazwyczaj jednak tak nie jest, trudno “trafić” w dobry stan z cechami, przez których przejście jest niezbędne do finalnego rozwiązania.

W dalszym ciągu jednak rozwiązanie z wykorzystaniem proponowanych heurystyk “zwycięża czasowo” nad tym bez, lecz nie zawsze (z powodów wymienionych w poprzednim przypadku). Im bardziej skomplikowany problem, tym różnice są większe. Oczywiście jest także różnica w stosunku do wyników algorytmów w prostszym przypadku, bez żadnych podcelów - znalezienie rozwiązania jest niemal zawsze szybsze.

Domena: DINNER * BFS (bez heurystyki): Razem 46 kroków w 4.0797s * **A*** (z heurystyką): Razem 46 kroków w 2.9528s * **Przyspieszenie A* względem BFS:** 1.38x

Domena: MAGICWORLD * BFS (bez heurystyki): Razem 41 kroków w 6.3687s * **A*** (z heurystyką): Razem 41 kroków w 3.4367s * **Przyspieszenie A* względem BFS:** 1.85x

Domena: BLOCKSWORD * BFS (bez heurystyki): Razem 16 kroków w 24.5962s * **A*** (z heurystyką) Razem 16 kroków w 0.0464s * **Przyspieszenie A* względem BFS:** 530.22x

Przyspieszenie, otrzymane przez korzystanie z heurystyki zależy w dużym stopniu od samego problemu - jak widać powyżej w problemach, które raczej analizują problemy binarne (tak jak domena dinner) korzyść czasowa jest mniejsza, niż w bardzo rozgałęzionych światach, z wieloma cechami, które mogą mieć wiele wartości (np. na czym stoi dany blok). W takich przypadkach korzystanie z heurystyk jest praktycznie wymagane, aby dostać rozwiązanie problemu w skończonym czasie.